WEEK 01

Values, types and execution

Python foundations

Values, types and execution

Names, values and statements

Values have types

Numbers support arithmetic; quoted values are text.

```
print(3 + 4)       # 7
print("3" + "4")   # 34
```

Names refer to values

Assignment evaluates the right side, then binds the name.

```
count = 3
count = count + 1  # now 4
```

Statements run in order

An earlier result does not recalculate when an input changes.

```
count = 3; total = count * 2
count = 4; print(total)  # 6
```

A written total keeps its old value

Write the shopping note

`bags = 4`

`total = 4 * 3`

12

Change the bags

`bags = 2`

total is still

12

Calculate again

`bags = 2`

`total = 2 * 3`

6

```
bags = 4
total = bags * 3
bags = 2
print(total, bags * 3)
```

Changing an input does not rerun an earlier calculation.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
name = value	Assign a value	Keep an input for later
3 * 4	Multiply numbers	Calculate a quantity
print("Total:", total)	Display a value	Label output and units
# explanation	Comment; not executed	Explain an assumption

Python Basics · sections 00, 01, 02, 03, 14 · use only when you need a lookup.

Names and text are different

```
count = 4
unit = 3
total = count * unit
print(count)
print("count")
print(type(count).__name__)
print(total)
```

1. A name retrieves its current value.
2. Quotes make text, not a variable lookup.
3. Expected: 4, count, int, 12.

Variables and string literals

```
01 containers = 4
02 print(containers)
03 print("containers")
04 print(type(containers).__name__)
```

1. Predict all three outputs before running
2. Assignment associates a name with a value
3. Change the value and predict again

Quotes create text; an unquoted name retrieves its assigned value.

Numbers and strings behave differently

```
01 print(3 * 4)
02 print("3" * 4)
03 assert 3 * 4 == 12
```

1. How does quoting the first value change the output?
2. Container counts use integer values
3. Keep expectations independent of output

Valid code can still express the wrong operation.

Assign, calculate and print

```
description = "Practice meal containers"
portions_per_container = 3
containers = 4
planned = portions_per_container * containers
print(description)
print("Containers:", containers)
print("Planned portions:", planned)
assert planned == 12
```

1. Assign input values first.
2. Use an expression to calculate a result.
3. print() displays the stored result.

Keep inputs, calculation and display as separate statements.

Execution order: when is a value recalculated?

```
01 portions_per_container = 3
02 containers = 4
03 planned = portions_per_container * containers
04 containers = 2
05 print("Before:", planned)
06 planned = portions_per_container * containers
07 print("After:", planned)
08 assert planned == 6
```

1. Predict the value printed before recalculation
2. Then predict the value printed after recalculation
3. Rerun the calculation after changing inputs

Changing an input does not rerun an earlier multiplication.

One language, two execution environments

Notebook: .ipynb

Narrative and code cells share a persistent kernel. Run cells and inspect their results.

Source file: .py

A new interpreter reads saved statements from top to bottom. Its output appears in a terminal.

A new script process does not inherit notebook variables.

Save and run a Python file

Context	Command or observation
Working directory	learning_materials/
Terminal	python programs/first_pickup.py
Choose Python	Use your course interpreter; see programs/README.md
Default calculation	3 portions/container × 4 containers
Expected meaning	12 planned portions, not delivered impact

Replace python with your course interpreter; save before rerunning.

Hands-on practice

Week_01.ipynb · Session 2

Python practice

Predict values

Distinguish count from "count".
Predict the type and product.

Change and observe

Change count after the calculation.
Recalculate the stored total.

Repair a type bug

Replace the quoted numeric input.
Check both required counts.

Complete these three core tasks. Saved-file execution is optional.

Project studio

Labelled planning calculation

The project need

1

Need

The coordinator asks: how much are we planning?

2

Operation

Use the declared counts to calculate and label planned portions.

3

Result

The coordinator can read the estimate and ask about unclear units.

4 containers × 3 = 12 planned portions.

Trace a declaration into a planning note

1

Declared input

3 planned portions in each specified container

2

Proposed count

4 containers in this practice plan

3

Calculation

3 multiplied by 4 gives 12 planned portions

4

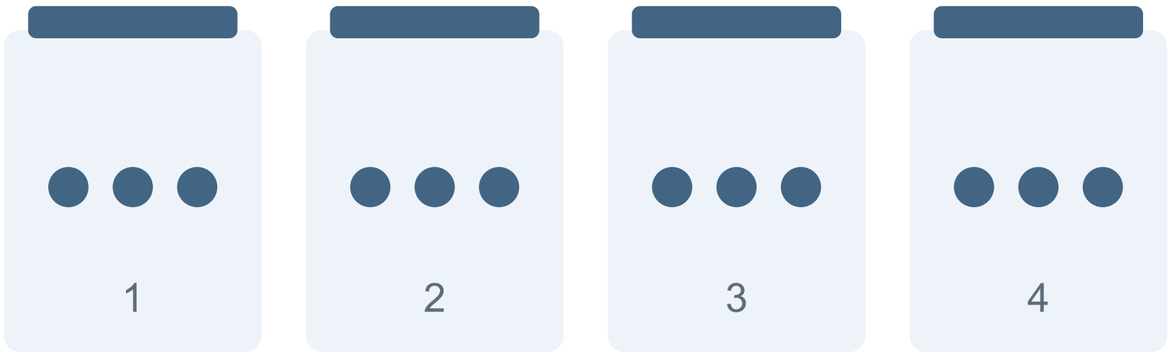
Labelled output

Show the units and retain planning status

Portions per container multiplied by containers gives planned portions.

See the values behind the calculation

4 containers



3 declared portions per container

3 × 4 = 12

Planned portions

A calculation from declared inputs.

Project task

Supplied starting point

A notebook starter provides variable names and print calls.

You implement / do

Set description, portions_per_container=2 and containers. Calculate and display planned_portions.

Check the evidence

Run containers 1, 3 and 0: expect 2, 6 and 0. Explain why this is a plan, not recorded delivery.

Explain the input units, the calculation and one coordinator question.

Project checks

Step	What to show
Implementation	Print description, input counts and planned portions.
Checks	2 portions/container × 1, 3, 0 containers → 2, 6, 0.

Explain the input units, the calculation and one coordinator question.

Further reading in English

1. ## Python tutorial: numbers and strings

   https://docs.python.org/3/tutorial/introduction.html

2. ## Python tutorial: input and output

   https://docs.python.org/3/tutorial/inputoutput.html

Read one relevant section, then bring an example and a question.